Introduction to React Native

Build Native Mobile Apps with JavaScript and React

SoftUni Team

Technical Trainers

Software University

SoftUni

Software University

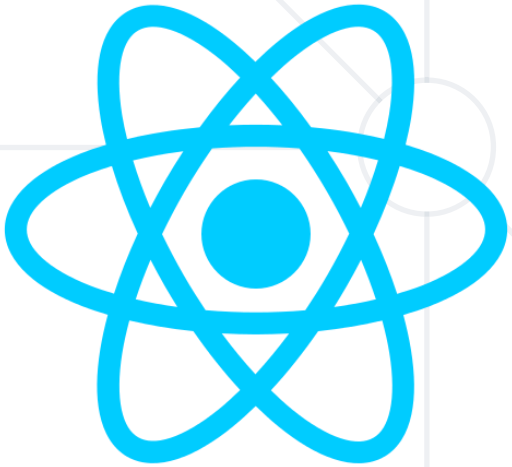
https://softuni.bg

Table of Contents

React Philosophy

React Recap

- Declarative **UI**
- **Component-based** architecture
- Platform **independent**
- **Unidirectional** data flow
- State → UI



Learn Once, Write Anywhere

- Same mental **model**:
  - Components
  - Hooks
  - State management
  - vDOM

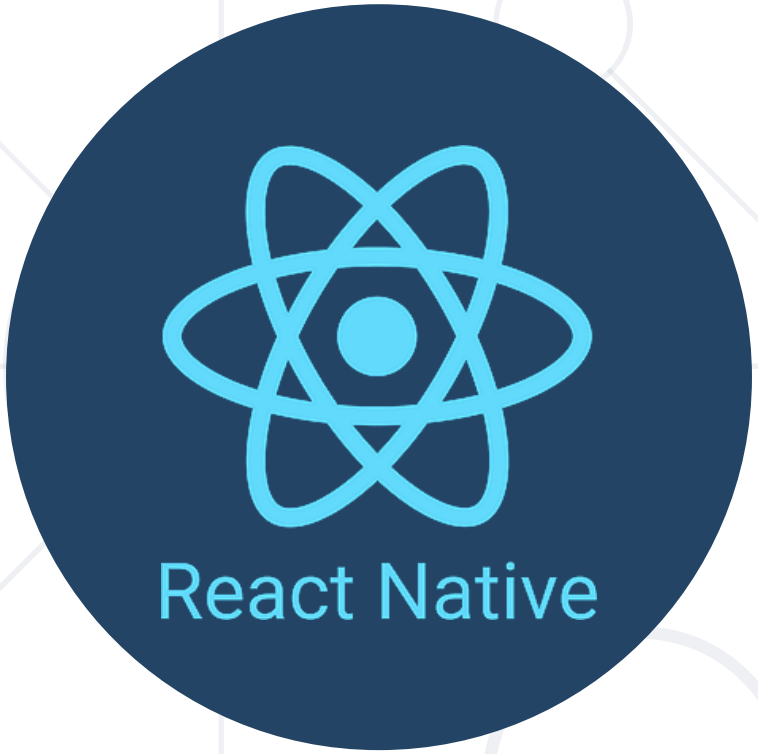
What is a React renderer?

- A **renderer** is the part of React that:
  - Takes the React component tree
  - Translates it into a specific target environment
- React itself (react) is **platform-agnostic**
- Renderers are what make React work on **different platforms**

React Renderers

Software University

- Web (**react-dom**)
- Mobile (**react-native**)
- Server (**react-dom/server**)
- Custom
  - **React Three Fiber** (3D / WebGL)
  - **Ink** (CLI apps)
  - **React PDF**



React Native Intro

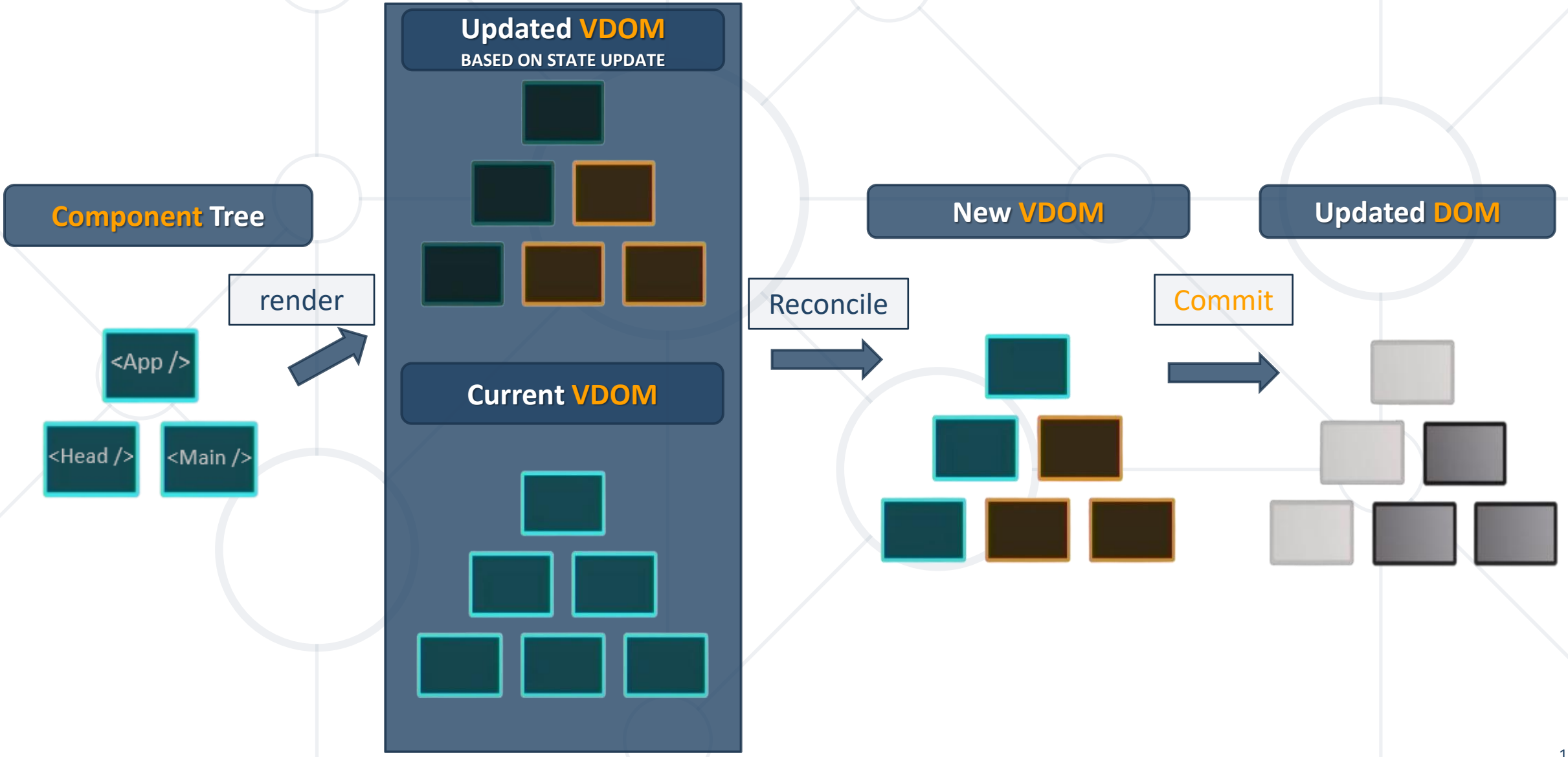
What Is React Native?

- **React Native** is a framework for building **mobile apps** using **JavaScript and React**, allowing you to create **native iOS and Android apps** from a single codebase
  - JavaScript framework for **mobile apps**
  - Uses React concepts
  - Collection with special react components
  - React renderer for **native UI components**
  - Native platform APIs exposed to JavaScript

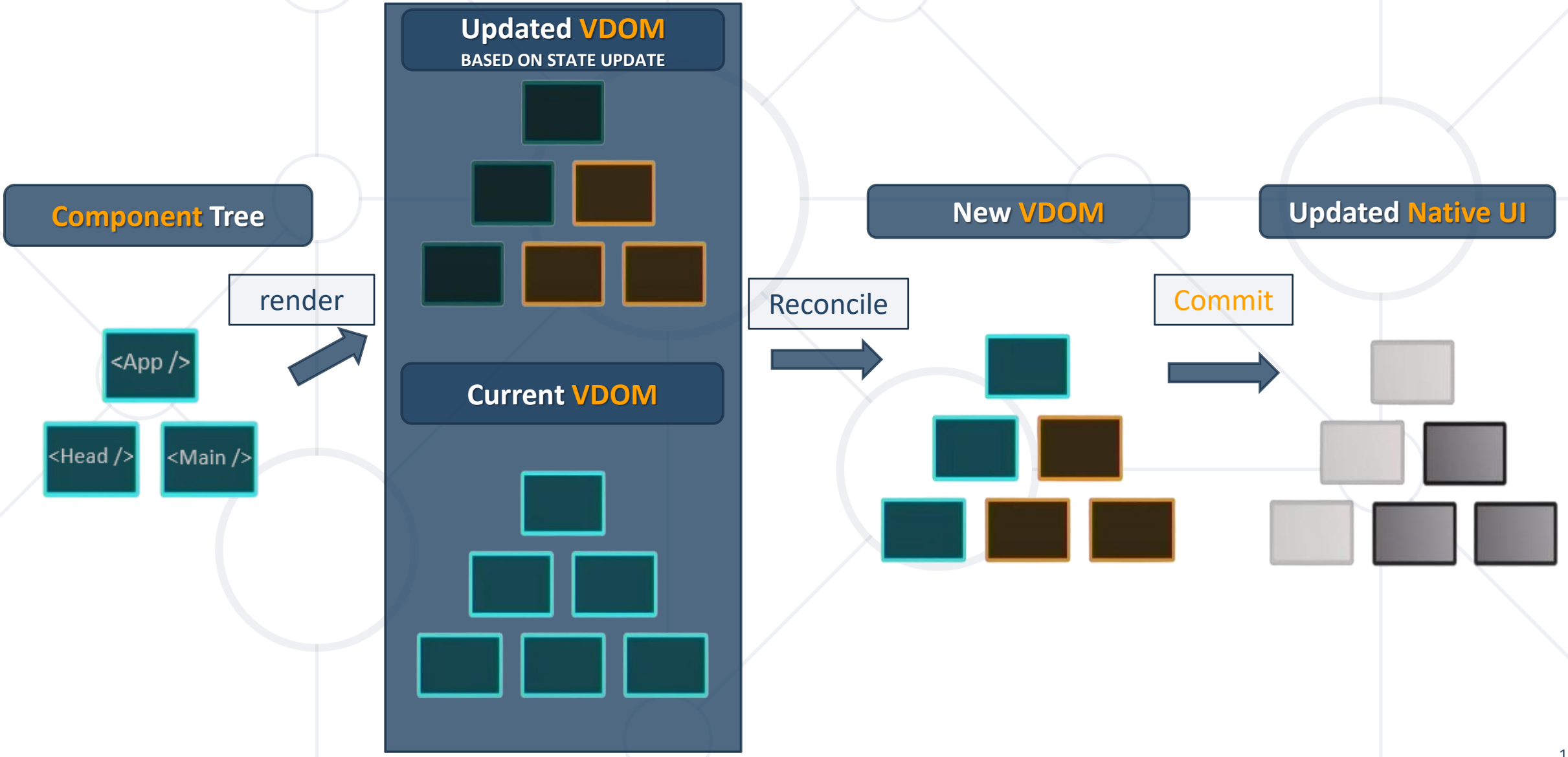
React Web vs React Native

- Same **React** philosophy and component-based architecture
- **Different** rendering targets and environments
- React Web → renders to **Browser DOM**
- React Native → renders to **Native UI components**
- Shared developer experience, but platform-specific performance and APIs

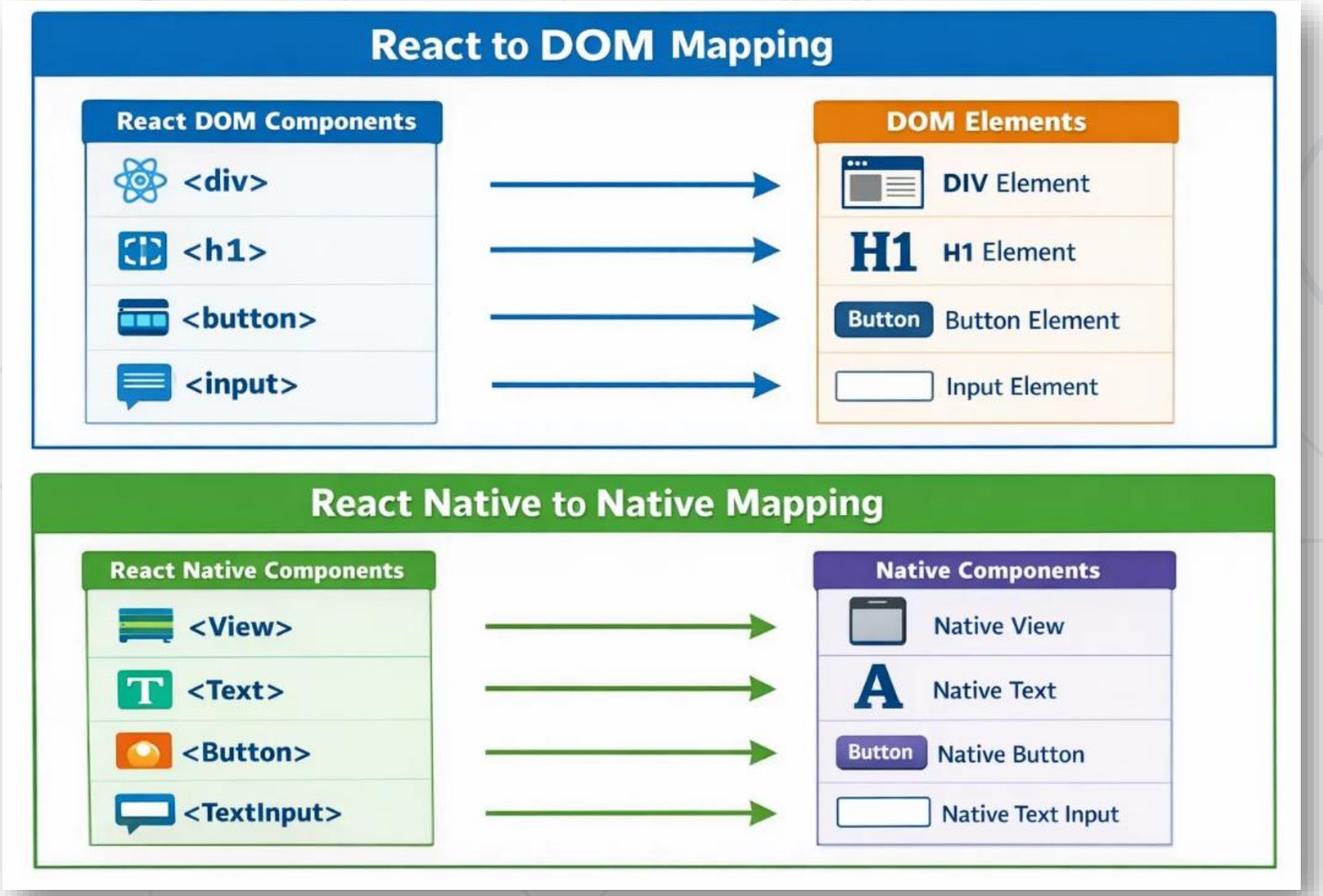
React Flow in React Web



React Flow in React Native



Component Mappings



Native Components

Software University

React Native Components → Native Equivalents

React Native Component	iOS Native Component	Android Native Component
View	UIVew	android.view.View
Text	UILabel / NSTextView	android.widget.TextView
Image	UIImageView	android.widget.ImageView
ScrollView	UIScrollView	android.widget.ScrollView
TextInput	UITextField / UITextView	android.widget.EditText

Platforms

- iOS
- Android
- Web
- Windows
- macOS

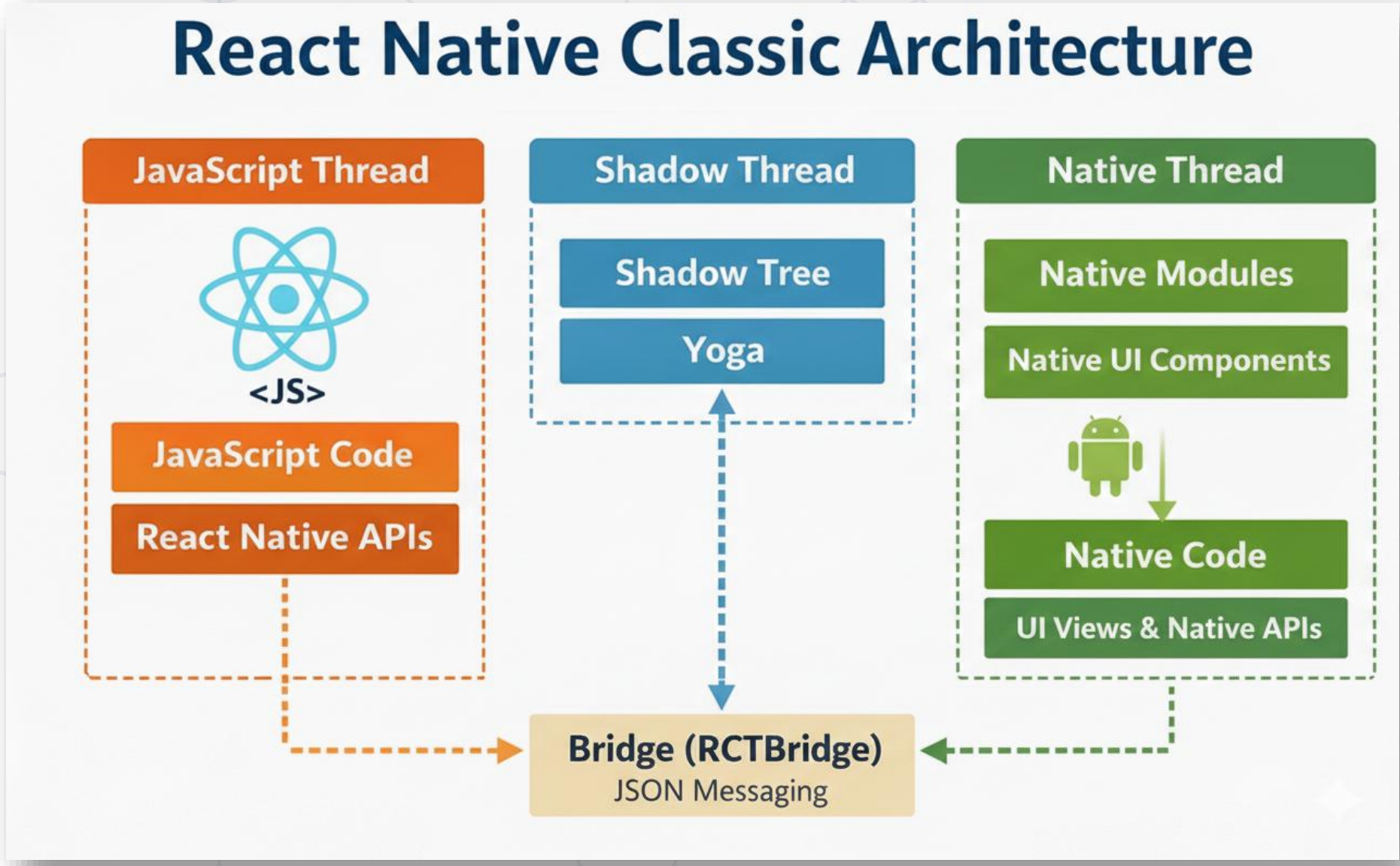
React Native Rendering Model

How React Native Works

Software University

- React Native runs on multiple threads
  - **JS Thread**
    - Executes JavaScript
    - Runs React logic (render, hooks, state)
    - Single Threaded
  - **UI Thread**
    - Runs native code
    - Renders actual native views
    - Handles layout, touch events, gestures

Classic Architecture



Bridge

Software University

- **Bridge** is a communication layer between JS and native
- Uses async communication
  - **JSON** serialization and deserialization
- Slow rendering
- JS could not communicate with native modules **directly** (always over the bridge)

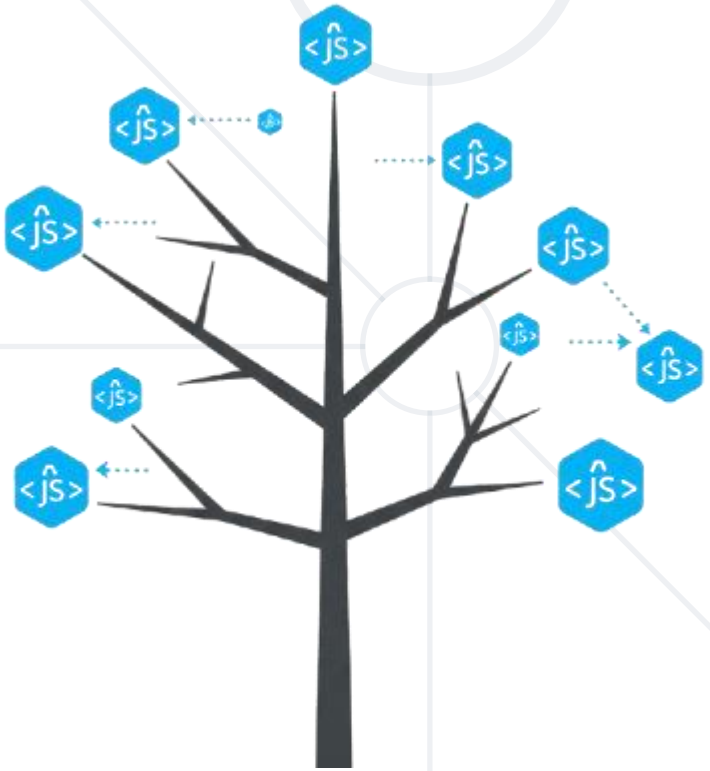
Yoga

- **Yoga** is the layout engine used by React Native to calculate Flexbox-based UI layouts
  - A **C++ layout engine**
  - Implements **Flexbox**
  - Platform-independent (used on **iOS** and **Android**)
  - Outputs layout sizes and positions for native views
  - Runs on **C++**

Shadow Tree

Software University

- **Virtual Native Tree** Data Structure (similar to DOM Tree)
  - A **platform-agnostic** representation of the UI
  - Contains: layout info, hierarchy, styles
  - No actual native views

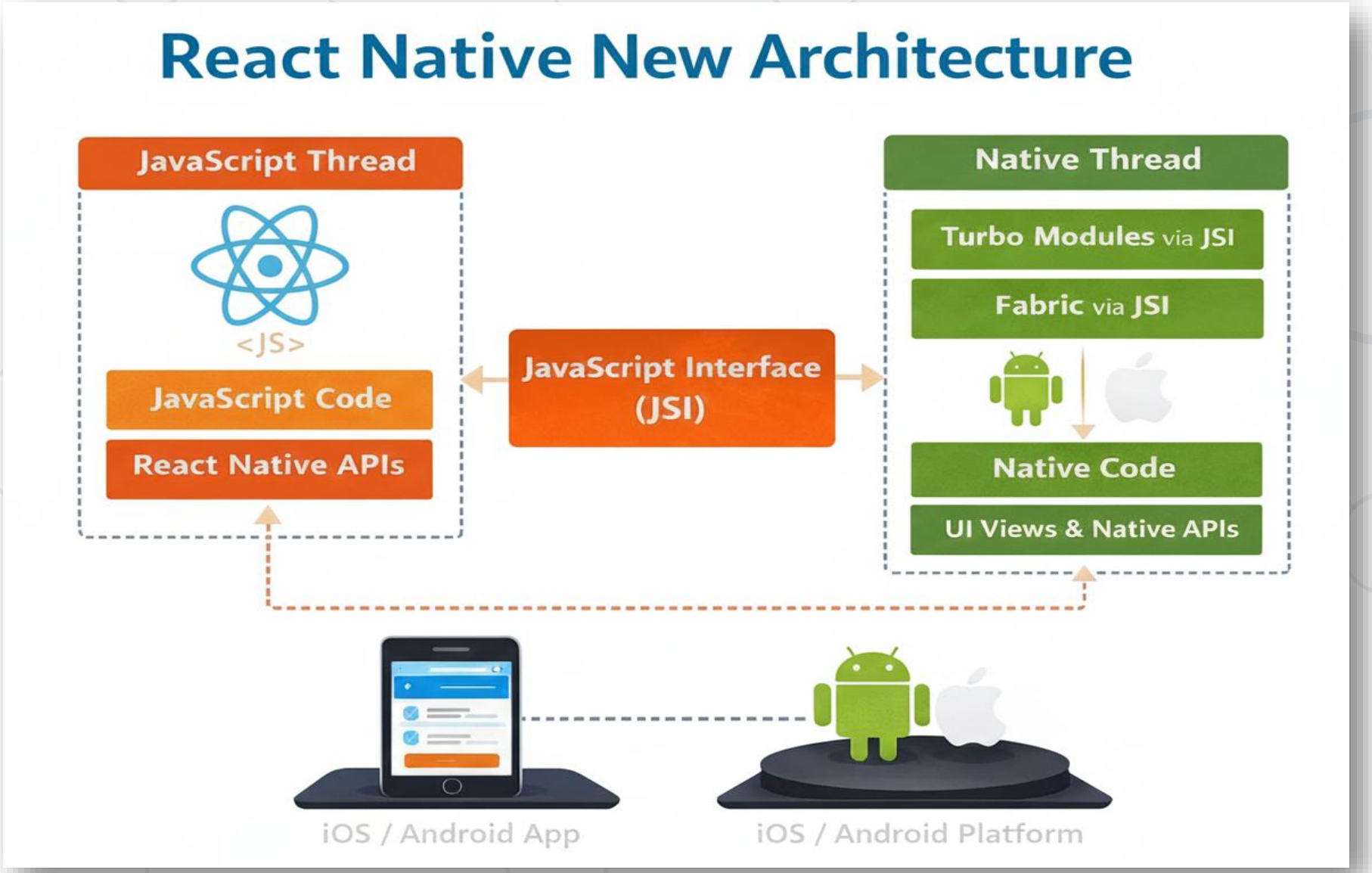


Shadow Thread

Software University

- The **Shadow Thread** is where React Native calculates UI layout
  - Computes layout using **Yoga** layout engine
  - Builds a shadow view tree
  - Passes layout results to the UI thread over the bridge
  - Native thread but **separate** from UI Thread

New Architecture



JSI (JavaScript Interface)

- **JSI (JavaScript Interface)** is a bidirectional interface that lets JavaScript talk directly to native code without using the bridge.
  - Enables direct, synchronous calls
  - Written in **C++**
  - Improves **performance** and **startup** time
  - Foundation for **Fabric and TurboModules**

Fabric

Software University

- **Fabric** is React Native's new rendering system that makes UI updates faster and more reliable.
  - Runs in **UI Thread**, implemented in C++
  - Synchronous rendering
  - Better performance and Smoother interactions
  - No more bridge communication no JSON communication
  - Uses JSI (JavaScript Interface) (direct JS ↔ Native)
  - **Shadow Tree** and **Yoga** are part of the **Fabric** renderer

Turbo Modules

- **TurboModules** are a faster native module system that lets JavaScript access native APIs efficiently.
  - Built on **JSI**
  - Supports **lazy loading**
  - Type-safe
  - Better performance

Expo

Full-Stack React Native framework

What is Expo?

Software University

- **Expo** is a set of tools and services that makes building, running, and deploying React Native apps faster and easier.
  - No native setup needed
  - Built-in device APIs
  - Instant app preview
  - Easy app publishing

Why use Expo?

Software University

- **Why use Expo?**
  - Faster setup and development
  - No native code configuration
  - No platform specific code (**Android & iOS**)
  - Built-in APIs out of the box
  - Smooth build and release process

Just start building and worry for configuration later!

Expo Workflows

Managed Workflow

- Zero native code
- Preconfigured APIs
- Best for start, learning and prototyping
- Ejectable

Bare Workflow

- Full native control
- Similar to React Native CLI
- For advanced / custom native needs

We start with Managed. You can always "eject" later.

Expo Go

Software University

- A **mobile app** for running your project instantly on real device.
  - No local build needed
  - Live reload on save
  - Works via QR code
  - Great for fast testing
  - No emulators needed

Your phone becomes your dev environment.

Built-in APIs

Expo gives you easy access to:

- Camera
- Media library
- Location
- Sensors
- Haptics
- Secure storage

Without Expo

- Native setup
- Platform differences
- Extra configuration

Just import it and use it!

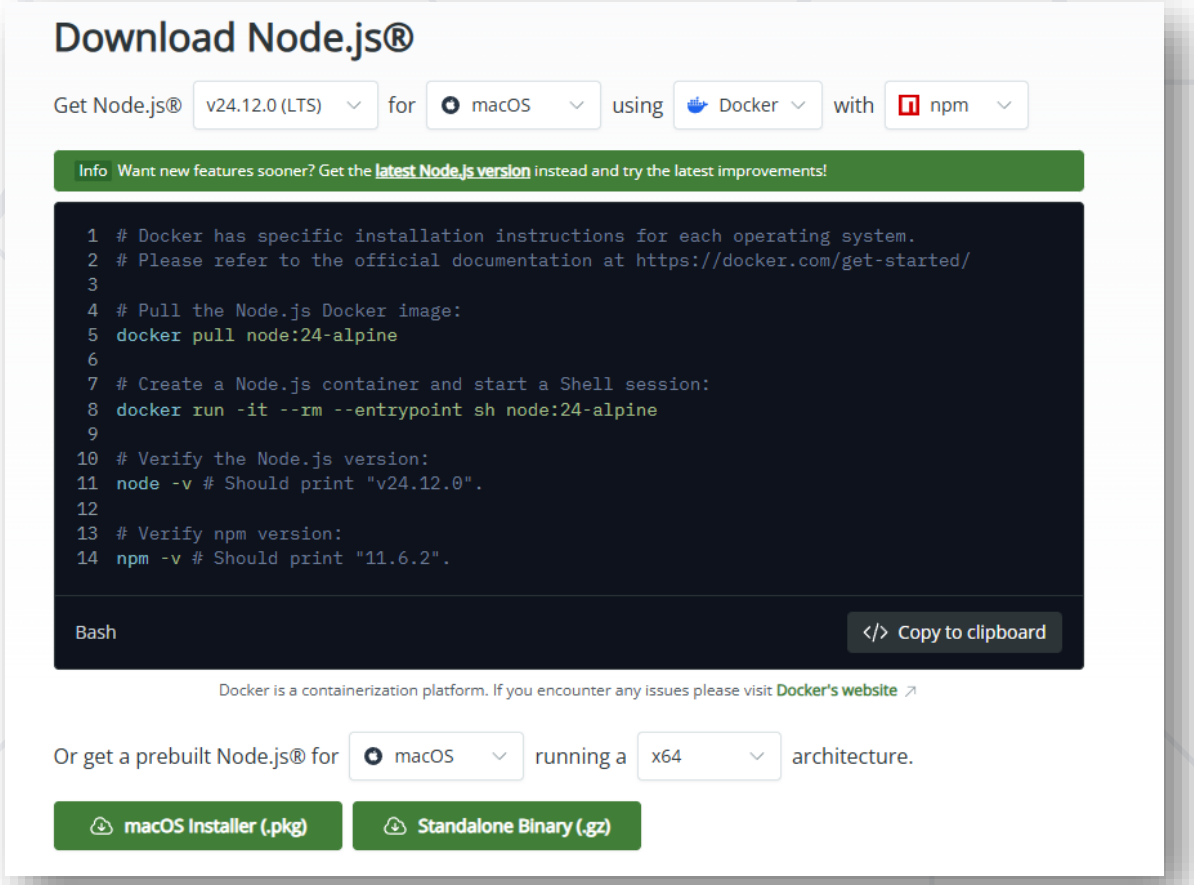
Developer Environment

Install Node.js

Software University

- **Node.js** is required to run the JavaScript environment and essential development tools outside the browser
- Includes **npm**, which you will use to install project dependencies
- Powers the **Metro Bundler**, the engine responsible for compiling your code and serving it to your simulator or device

https://nodejs.org/en/download/

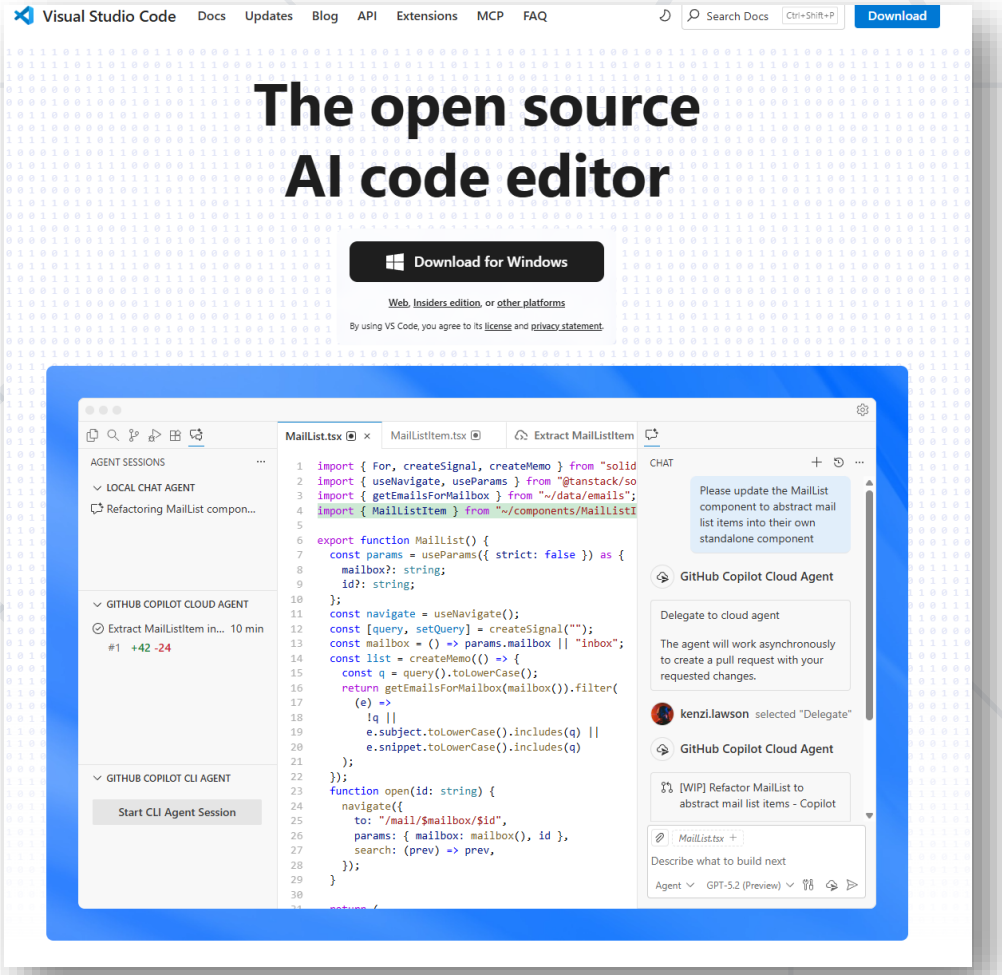


Install Node.js

Software University

- The **industry-standard** code editor for React Native, offering a powerful environment for mobile development
- Supports essential extensions like **ES7+ React/Redux/React-Native snippets** and **Prettier** for faster, cleaner coding
- Features a built-in terminal and debugging tools that allow you to manage your packager and inspect code without leaving the editor

https://code.visualstudio.com/download



Install Expo Go

- Run and test your app directly on a physical device by scanning a QR code, bypassing the need for complex mobile SDKs or cables
- Streamlines development by removing the requirement for Android Studio or Xcode, allowing you to focus entirely on writing React code

Official Website:	https://expo.dev/go
Android:	Google Play
iOS:	App Store

Metro

- **Metro** is the React Native bundler that builds and serves your app's JavaScript.
  - Resolves imports
  - Transpiles code
  - Serves the bundle
  - Enables fast refresh

Official Website: https://metrobundler.dev/

Create First Project

- Use a single command to generate a **pre-configured project** structure with all necessary dependencies

```
npx create-expo-app@latest first-project
```

- **Launch** your app in a browser or on your phone to see changes in real-time

```
cd first-project
```

- **Run** to develop quickly in a browser tab

```
npm run web
```

Run on Physical Device

- Start project

```
npm start
```

- Scan QR Code to open in physical device

Core Components

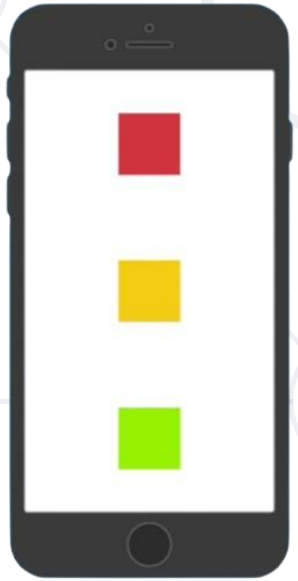
(View, Text, Image)

Text

- Displays **text**, like paragraph element
  - Shows text on screen
  - Supports styling
  - Can be nested
  - Supports touch handling

View

- A container for layout and structure, like div element
  - **Wraps** UI elements
  - Supports **flexbox** layout
  - Used for **styling** and **positioning**



Image

- Displays images in an app
  - Import image as static resource

```
<Image source={require('./assets/favicon.png')} />
```

  - Import image as network resource

```
<Image source={{
uri: 'https://reactnative.dev/img/tiny_logo.png’,
width: 64,
height: 64,
 }} />
```

Button

- Triggers an **action** when pressed
  - Handles user taps
  - Displays a label
  - Executes a function

```
<Button
title="Press Me"
onPress={() => alert('Button Pressed!’)}
 />
```

Debugging

Hands-on Debugging: Tools & Techniques in Action

Summary

- React is declarative, component-based, and platform-agnostic
- React Native uses the React mental model with native UI components for iOS and Android
- Expo simplifies development with fast setup, built-in native APIs, and easy device testing
- Core components: View (layout), Text (text rendering), Image (images)

Trainings @ Software University (SoftUni)

- Software University – High-Quality Education, Profession and Job for Software Developers
  - softuni.bg
- Software University Foundation
  - softuni.foundation
- Software University @ Facebook
  - facebook.com/SoftwareUniversity

License

- This course (slides, examples, demos, exercises, homework, documents, videos and other assets) is **copyrighted content**
- Unauthorized copy, reproduction or use is illegal
- © SoftUni – https://about.softuni.bg/
- © Software University – https://softuni.bg